EngAInOS Gate Board Instructions

Location:

```
/home/mytruelove/Desktop/burdens_of_a_forgotten_past/EngAIn/engainos/gates/
GATE_BOARD_INSTRUCTIONS.md
```

Purpose:

This file separates EngAInOS gates into two proof classes:

```
MIGRATION GATES
  Prove old code crossed from historical/mixed folders into clean EngAInOS lanes
safely.

SYSTEM CONTRACT GATES
  Prove the active system obeys EngAInOS authority law after migration.
```

Core doctrine:

```
Migration gates prove how a file moved.

System contract gates prove the active system is still safe.

Adapters convert.
Gates judge.
Relays carry approved calls.
Runtime executes.
```

Do not treat all gates as the same kind of proof.

A migration gate may become historical after the move is accepted.

A system contract gate stays active and must keep passing.

---

1. Current Root

All commands assume:

```
cd /home/mytruelove/Desktop/burdens_of_a_forgotten_past/EngAIn
```

Never use Downloads as a working directory.

Never use `engainos/` as a Godot project root.

Never use `godotsim/` as a Godot project root.

If Godot needs a root later, use a dedicated presentation folder such as:

```
presentation/godot/
```

---

2. Migration Gates

Migration gates prove old historical code was moved, split, or preserved safely.

These gates should live under:

```
engainos/gates/migration/
```

until the folder split is performed.

For now, if they still live directly in `engainos/gates/`, run them from the repo root.

2.1 AP Core Migration Authority

Gate file:

```
engainos/gates/gate_ap_core_migration_authority.py
```

Command:

```
python engainos/gates/gate_ap_core_migration_authority.py
cat scratch/ap_core_migration_report.json
```

Historical source file:

```
godotengain/engainos/core/ap_core.py
```

Migrated destination file:

```
engainos/core/ap_core.py
```

Report:

```
scratch/ap_core_migration_report.json
```

Proof class:

```
MIGRATION
```

Proves:

```
Historical ap_core.py exists.
Destination was controlled.
Imports stay inside EngAInOS authority/core boundary.
No Godot bridge/render/spawn language is present.
No forbidden runtime side-effect calls are present.
ap_core.py is classified as ENGAINOS_CORE_AUTHORITY.
```

Expected final state:

```
acceptance: ACCEPTED
classification: ENGAINOS_CORE_AUTHORITY
authority_change: no
behavior_change: no
```

2.2 AP Core Schema-Correct Behavior Parity

Gate file:

```
engainos/gates/gate_ap_core_behavior_parity.py
```

Command:

```
python engainos/gates/gate_ap_core_behavior_parity.py
cat scratch/ap_core_behavior_parity_report.json
```

Historical source file:

```
godotengain/engainos/core/ap_core.py
```

Migrated destination file:

```
engainos/core/ap_core.py
```

Report:

```
scratch/ap_core_behavior_parity_report.json
```

Proof class:

```
MIGRATION
```

Proves:

```
Old and new ap_core.py behave identically using the real AP schema.

Negative health schema:
  delta.entities.<entity_id>.health

Duplicate guard schema:
  delta.guards.<location_id> = <guard_id>
```

Required test cases:

```
1. Reject negative health:
   snapshot.entities.hero.health = 10
   delta.entities.hero.health = -1

2. Reject duplicate guard:
   delta.guards.north_gate = guard_001
   delta.guards.south_gate = guard_001
```

```
3. Allow valid delta:
   delta.entities.hero.health = 9
   delta.guards.north_gate = guard_001
   delta.guards.south_gate = guard_002
```

Expected final state:

```
acceptance: ACCEPTED
signal_quality: SIGNAL
old_signal: true
new_signal: true
parity_passed: true
```

Important note:

The earlier flat-payload parity gate was non-signal because it used:

```
{"health": -1}
{"guard": true, "double_guard": true}
```

Those are not the AP schema read by `ap_core.py`.

Correct stamp:

```
AP_CORE_FLAT_PAYLOAD_PARITY: TRUE_BUT_NON_SIGNAL
AP_CORE_SCHEMA_CORRECTED_BEHAVIOR_PARITY: TRUE
```

2.3 AP Engine Migration Authority

Gate file:

```
engainos/gates/gate_ap_engine_migration_authority.py
```

Command:

```
python engainos/gates/gate_ap_engine_migration_authority.py
cat scratch/ap_engine_migration_authority_report.json
```

Historical source file:

```
godotengain/engainos/core/ap_engine.py
```

Current root file:

```
engainos/core/ap_engine.py
```

Report:

```
scratch/ap_engine_migration_authority_report.json
```

Proof class:

```
MIGRATION
```

Proves:

```
Historical ap_engine.py is safe enough for review.
Root ap_engine.py default-rule bootstrap is preserved.
Root ap_engine.py must not be overwritten.
Historical ap_engine.py is not a direct replacement for root ap_engine.py.
```

Expected final decision:

```
decision: ACCEPT_INTENTIONAL_MERGE_REVIEW
overwrite_allowed: false
```

Important result:

Historical `ap_engine.py` was split instead of merged.

2.4 AP Engine Historical Probe

Gate file:

```
engainos/gates/gate_ap_engine_historical_probe.py
```

Command:

```
python engainos/gates/gate_ap_engine_historical_probe.py
cat scratch/ap_engine_historical_probe_report.json
```

Historical source file:

```
godotengain/engainos/core/ap_engine.py
```

Root file checked for preservation:

```
engainos/core/ap_engine.py
```

Report:

```
scratch/ap_engine_historical_probe_report.json
```

Proof class:

```
MIGRATION
```

Proves:

```
Historical AP engine symbols are present.
Root AP default bootstrap symbols are present.
Historical load_rules_from_scene accepts path-shaped scene inputs.
```

Historical symbols:

```
APInternalRule
StateProvider
ZWAPEngine
load_rules_from_scene
```

Root bootstrap symbols:

```
DEFAULT_RULES_REGISTERED
register_default_rules
build_default_ap_system
```

```
check_default_ap
is_default_valid
```

Expected final state:

```
acceptance: ACCEPTED_FOR_MERGE_DESIGN
merge_allowed: false
overwrite_allowed: false
```

2.5 AP Engine Historical Source Map

Gate file:

```
engainos/gates/gate_ap_engine_historical_source_map.py
```

Command:

```
python engainos/gates/gate_ap_engine_historical_source_map.py
cat scratch/ap_engine_historical_source_map_report.json
```

Historical source file:

```
godotengain/engainos/core/ap_engine.py
```

Report:

```
scratch/ap_engine_historical_source_map_report.json
```

Proof class:

```
MIGRATION
```

Proves:

```
Historical source bodies were extracted and reviewed.
ZWAPEngine is stateful.
ZWAPEngine can execute ticks.
```

```
ZWAPEngine can append ZON timeline events.
StateProvider mutates internal state.
```

Resulting decision:

```
Do not merge historical ZWAPEngine into root engainos/core/ap_engine.py.
Split historical runtime-style AP engine into:
  engainos/core/ap_zw_engine.py
```

2.6 AP ZW Engine Migration

Gate file:

```
engainos/gates/gate_ap_zw_engine_migration.py
```

Command:

```
python engainos/gates/gate_ap_zw_engine_migration.py
cat scratch/ap_zw_engine_migration_report.json
```

Historical source file:

```
godotengain/engainos/core/ap_engine.py
```

Migrated split file:

```
engainos/core/ap_zw_engine.py
```

Root bootstrap file preserved:

```
engainos/core/ap_engine.py
```

Report:

```
scratch/ap_zw_engine_migration_report.json
```

Proof class:

```
MIGRATION
```

Proves:

```
ap_zw_engine.py preserves historical ZW AP symbols.
Root ap_engine.py default bootstrap remains intact.
ap_zw_engine.py has no Godot/server/render/network subprocess imports.
load_rules_from_scene reads a JSON path and returns a rules dict.
StateProvider mutates only its internal state container in probe.
execute_tick timeline writing is fenced by explicit enable_timeline_write.
```

Expected final state:

```
acceptance: ACCEPTED
migration_acceptance: ACCEPTED
runtime_wiring_allowed: true
merge_into_root_ap_engine_allowed: false
overwrite_allowed: false
requires_fence_before_runtime: false
```

2.7 AP ZW Engine Canonical Claim

Gate file:

```
engainos/gates/gate_ap_zw_engine_canonical_claim.py
```

Command:

```
python engainos/gates/gate_ap_zw_engine_canonical_claim.py
cat scratch/ap_zw_engine_canonical_claim_report.json
```

Checked file:

```
engainos/core/ap_zw_engine.py
```

Report:

```
scratch/ap_zw_engine_canonical_claim_report.json
```

Proof class:

```
MIGRATION / PROVENANCE
```

Proves:

```
ap_zw_engine.py no longer falsely claims CANONICAL_AP_ENGINE_MODULE.
Legacy source provenance is preserved.
Migrated module identity is declared.
Root ap_engine.py authority is not stolen.
```

Required constants:

```
LEGACY_SOURCE_AP_ENGINE_MODULE = "godotengain.engainos.core.ap_engine"
MIGRATED_AP_ZW_ENGINE_MODULE = "engainos.core.ap_zw_engine"
```

Forbidden constant:

```
CANONICAL_AP_ENGINE_MODULE
```

Expected final state:

```
acceptance: ACCEPTED
has_CANONICAL_AP_ENGINE_MODULE: false
```

---

3. System Contract Gates

System contract gates prove the active system obeys current EngAInOS authority law.

These gates should live under:

```
engainos/gates/contracts/
```

until the folder split is performed.

For now, if they still live directly in `engainos/gates/`, run them from repo root.

3.1 AP Default Rules Registered

Gate file:

```
engainos/gates/gate_ap_default_rules_registered.py
```

Command:

```
python engainos/gates/gate_ap_default_rules_registered.py
cat scratch/ap_default_rules_registered_report.json
```

Files checked:

```
engainos/core/ap_core.py
engainos/core/ap_engine.py
```

Report:

```
scratch/ap_default_rules_registered_report.json
```

Proof class:

```
SYSTEM_CONTRACT
```

Proves:

```
Root AP default rules are declared.
Default AP engine imports no bridge/render/server modules.
Default AP rejects negative health.
Default AP rejects duplicate guard assignment.
Default AP allows valid AP delta.
```

Rules:

```
no_negative_health
no_double_guard
```

Expected final state:

```
acceptance: ACCEPTED
GATE_REJECTS_NEGATIVE_HEALTH: TRUE
GATE_REJECTS_DOUBLE_GUARD: TRUE
GATE_ALLOWS_VALID_DELTA: TRUE
```

3.2 AP ZW Engine Timeline Fence

Gate file:

```
engainos/gates/gate_ap_zw_engine_timeline_fence.py
```

Command:

```
python engainos/gates/gate_ap_zw_engine_timeline_fence.py
cat scratch/ap_zw_engine_timeline_fence_report.json
```

File checked:

```
engainos/core/ap_zw_engine.py
```

Report:

```
scratch/ap_zw_engine_timeline_fence_report.json
```

Proof class:

```
SYSTEM_CONTRACT
```

Proves:

```
ZWAPEngine defaults to timeline-write disabled.
Dry-run execute_tick does not write timeline.jsonl.
Explicit enabled write uses explicit scratch root.
```

Expected final state:

```
acceptance: ACCEPTED
runtime_wiring_allowed: true
timeline_write_default: disabled
timeline_write_requires_explicit_enable: true
```

3.3 AP Runtime Repair Readiness

Gate file:

```
engainos/gates/gate_ap_runtime_repair_readiness.py
```

Command:

```
python engainos/gates/gate_ap_runtime_repair_readiness.py
cat scratch/ap_runtime_repair_readiness_report.json
```

File checked:

```
godotengain/engainos/core/ap_runtime.py
```

Report:

```
scratch/ap_runtime_repair_readiness_report.json
```

Proof class:

```
SYSTEM_CONTRACT
```

Proves:

```
Historical ap_runtime.py exists.
Dead code after unconditional return is gone.
All dispatched AP handlers are defined.
_handle_execute_tick references the timeline fence before execute_tick.
scenes_dir is no longer an unanchored relative default.
```

Expected final state:

```
acceptance: ACCEPTED_AS_REPAIR_READY
repair_ready: true
relay_creation_allowed: false
```

Important:

This gate is structural only. It does not prove behavior.

3.4 AP Runtime Behavior Probe

Gate file:

```
engainos/gates/gate_ap_runtime_behavior_probe.py
```

Command:

```
python engainos/gates/gate_ap_runtime_behavior_probe.py
cat scratch/ap_runtime_behavior_probe_report.json
```

File exercised:

```
godotengain/engainos/core/ap_runtime.py
```

Also uses:

```
engainos/core/ap_zw_engine.py
```

Report:

```
scratch/ap_runtime_behavior_probe_report.json
```

Proof class:

```
SYSTEM_CONTRACT
```

Proves real runtime bridge behavior:

```
ENGAIN_ROOT resolves to repo root.
ap_simulate_tick returns simulation result and does not write timeline.
ap_execute_tick without allow_execute is rejected.
ap_execute_tick with allow_execute executes without timeline write.
timeline write request is rejected unless runtime config enabled it.
timeline write occurs only when runtime and message both allow it.
ap_execution_history requires allow_history_read.
```

Expected final state:

```
acceptance: ACCEPTED_BEHAVIOR_PROVEN
behavioral_proof: true
relay_readiness_gate_allowed_next: true
relay_creation_allowed: false
```

3.5 AP Runtime Relay Readiness

Gate file:

```
engainos/gates/gate_ap_runtime_relay_readiness.py
```

Command:

```
python engainos/gates/gate_ap_runtime_relay_readiness.py
cat scratch/ap_runtime_relay_readiness_report.json
```

Files/reports checked:

```
godotengain/engainos/core/ap_runtime.py
scratch/ap_runtime_repair_readiness_report.json
scratch/ap_runtime_behavior_probe_report.json
```

Report:

```
scratch/ap_runtime_relay_readiness_report.json
```

Proof class:

```
SYSTEM_CONTRACT
```

Proves:

```
ap_runtime.py remains in godotengain and has not been copied into engainos/core.
AP runtime structural repair report is accepted.
AP runtime behavior proof is accepted.
Runtime imports fenced ZW engine.
Runtime contains execute, timeline, and history intent gates.
Relay MAY/MAY NOT contract is declared.
```

Expected final state:

```
acceptance: ACCEPTED_RELAY_READY
relay_creation_allowed: true
relay_file_allowed_next: engainos/relays/ap_runtime_relay.py
```

Relay doctrine:

```
Adapters convert.
Gates judge.
Relays carry approved calls.
Runtime executes.
```

Relay MAY:

```
load_or_import_repaired_APRuntimeIntegration
validate_caller_context_before_forwarding
forward_caller_supplied_AP_messages
```

```
reject_raw_runtime_or_player_input_without_acceptance_marker
return_runtime_response_without_declaring_truth
```

Relay MAY NOT:

```
instantiate_ZWAPEngine_directly
mutate_StateProvider_directly
load_scene_files_directly
write_timeline_directly
call_execute_tick_directly
manufacture_allow_execute_true
manufacture_enable_timeline_write_true
manufacture_allow_history_read_true
bypass_APRuntimeIntegration_intent_gates
declare_runtime_truth
```

3.6 AP Runtime Relay Behavior

Gate file:

```
engainos/gates/gate_ap_runtime_relay_behavior.py
```

Command:

```
python engainos/gates/gate_ap_runtime_relay_behavior.py
cat scratch/ap_runtime_relay_behavior_report.json
```

File checked:

```
engainos/relays/ap_runtime_relay.py
```

Report checked:

```
scratch/ap_runtime_relay_readiness_report.json
```

Report produced:

```
scratch/ap_runtime_relay_behavior_report.json
```

Proof class:

```
SYSTEM_CONTRACT
```

Proves relay source behavior using a fake runtime object:

```
Relay readiness report authorized relay creation.
Relay file exists.
Relay import boundary is clean.
Relay source does not bypass APRuntimeIntegration.
Relay source does not manufacture consent.
Relay rejects messages without engainos_accepted: true.
Relay forwards caller-supplied message without mutation.
Relay initialization forwards to APRuntimeIntegration.
```

Expected final state:

```
acceptance: ACCEPTED_RELAY_BEHAVIOR_PROVEN
relay_behavior_proven: true
relay_may_not_manufacture_consent: true
runtime_bridge_called_only_through_APRuntimeIntegration: true
```

Important:

This gate uses a fake runtime object for some checks.

It proves relay source discipline.

It does not prove real integration.

3.7 AP Runtime Relay Real Integration

Gate file:

```
engainos/gates/gate_ap_runtime_relay_real_integration.py
```

Command:

```
python engainos/gates/gate_ap_runtime_relay_real_integration.py
cat scratch/ap_runtime_relay_real_integration_report.json
```

Files exercised:

```
engainos/relays/ap_runtime_relay.py
godotengain/engainos/core/ap_runtime.py
engainos/core/ap_zw_engine.py
```

Report checked:

```
scratch/ap_runtime_relay_behavior_report.json
```

Report produced:

```
scratch/ap_runtime_relay_real_integration_report.json
```

Proof class:

```
SYSTEM_CONTRACT
```

Proves the full real path:

```
Accepted simulate message reaches real runtime through relay and writes no
timeline.
Unaccepted execute is rejected by relay before reaching runtime.
Accepted execute without allow_execute reaches real runtime and is rejected by
runtime intent gate.
Accepted execute with allow_execute runs through real runtime without timeline
write.
Real relay/runtime path rejects timeline write when runtime config is disabled.
Real relay/runtime path writes timeline only when runtime and caller both allow
it.
Real relay/runtime path requires allow_history_read for history access.
```

Expected final state:

```
acceptance: ACCEPTED_REAL_INTEGRATION_PROVEN
relay_real_integration_proven: true
```

```
relay_calls_real_APRuntimeIntegration: true
relay_does_not_manufacture_consent_in_real_path: true
timeline_write_real_path_requires_both_permissions: true
```

This gate closes the AP runtime relay thread.

---

4. Folder Separation Plan

Do not move files until the manifest exists.

Target layout:

```
engainos/gates/
  GATE_BOARD_INSTRUCTIONS.md
  GATE_MANIFEST.json

  migration/
    gate_ap_core_migration_authority.py
    gate_ap_core_behavior_parity.py
    gate_ap_engine_migration_authority.py
    gate_ap_engine_historical_probe.py
    gate_ap_engine_historical_source_map.py
    gate_ap_zw_engine_migration.py
    gate_ap_zw_engine_canonical_claim.py

  contracts/
    gate_ap_default_rules_registered.py
    gate_ap_zw_engine_timeline_fence.py
    gate_ap_runtime_repair_readiness.py
    gate_ap_runtime_behavior_probe.py
    gate_ap_runtime_relay_readiness.py
    gate_ap_runtime_relay_behavior.py
    gate_ap_runtime_relay_real_integration.py

  archive/
    non-signal gates
    superseded probes
    scratch-only experiments
```

Non-signal or superseded examples:

```
gate_ap_core_signature_probe.py
```

Reason:

```
It failed due module-qualified annotation class names, while real behavior
parity later proved the migration.
Keep it as archive/provenance, not active contract proof.
```

---

5. Verification Commands By Board

Run all active migration gates:

```
python engainos/gates/gate_ap_core_migration_authority.py
python engainos/gates/gate_ap_core_behavior_parity.py
python engainos/gates/gate_ap_engine_migration_authority.py
python engainos/gates/gate_ap_engine_historical_probe.py
python engainos/gates/gate_ap_engine_historical_source_map.py
python engainos/gates/gate_ap_zw_engine_migration.py
python engainos/gates/gate_ap_zw_engine_canonical_claim.py
```

Run all active system contract gates:

```
python engainos/gates/gate_ap_default_rules_registered.py
python engainos/gates/gate_ap_zw_engine_timeline_fence.py
python engainos/gates/gate_ap_runtime_repair_readiness.py
python engainos/gates/gate_ap_runtime_behavior_probe.py
python engainos/gates/gate_ap_runtime_relay_readiness.py
python engainos/gates/gate_ap_runtime_relay_behavior.py
python engainos/gates/gate_ap_runtime_relay_real_integration.py
```

Minimum AP acceptance board:

```
python engainos/gates/gate_ap_core_behavior_parity.py
python engainos/gates/gate_ap_default_rules_registered.py
python engainos/gates/gate_ap_zw_engine_migration.py
python engainos/gates/gate_ap_zw_engine_timeline_fence.py
python engainos/gates/gate_ap_runtime_behavior_probe.py
python engainos/gates/gate_ap_runtime_relay_real_integration.py
```

Expected board state:

```
AP_CORE_SCHEMA_CORRECTED_BEHAVIOR_PARITY: TRUE
AP_DEFAULT_RULES_REGISTERED: TRUE
AP_ZW_ENGINE_MIGRATION: ACCEPTED
AP_ZW_ENGINE_TIMELINE_FENCE: TRUE
AP_RUNTIME_BEHAVIORAL_PROOF: TRUE
AP_RUNTIME_RELAY_REAL_INTEGRATION: TRUE
```

---

6. Final AP Thread Status

Current accepted ladder:

```
engainos/core/ap_core.py
  mechanism
  schema-corrected parity proven

engainos/core/ap_engine.py
  default policy bootstrap
  AP default enforcement proven

engainos/core/ap_zw_engine.py
  fenced tick engine
  timeline write default disabled
  explicit write proven

godotengain/engainos/core/ap_runtime.py
  repaired historical runtime bridge
  structural + behavioral proof

engainos/relays/ap_runtime_relay.py
  accepted-call relay
  no consent manufacturing
  no runtime bypass
  real integration proven
```

Final AP status:

```
AP_CORE_THREAD: ACCEPTED
AP_ENGINE_THREAD: ACCEPTED
AP_ZW_ENGINE_THREAD: ACCEPTED
AP_RUNTIME_BRIDGE_REPAIR: ACCEPTED
AP_RUNTIME_RELAY_THREAD: ACCEPTED
```

Next possible target:

```
authority_gate.py
contract_validator.py
protocol_envelope.py
reality_mode.py
canon.py
engine_summary.py
```